

Entrevista Técnica — Loja ComprAqui (desenhe no draw.io)

Como funciona este exercício. Abaixo está uma **conversa** entre a equipe e o cliente do sistema **Loja ComprAqui** (dono de uma loja online). **Leia com atenção** e, a partir do que as pessoas dizem,

desenhe um diagrama de classes no draw.io (ou app.diagrams.net).

Não é para programar — é para **modelar**.

 **Seu diagrama precisa mostrar os quatro pilares da OO e os três tipos de relacionamento** (associação, agregação e composição). No fim há a **lista do que entregar**, um **lembrete da notação** e um **gabarito** (para o professor).

A entrevista

Gerente: Pessoal, vamos organizar o sistema da **Loja ComprAqui**. Vou trazer o cliente pra ele explicar, do jeito dele, o que o sistema precisa ter. Anotem tudo e depois a gente transforma em diagrama.

Cliente: No sistema **Loja ComprAqui**, a peça central é **Cliente**. Tem dois tipos: o **ClienteComum** (o comum) e o **ClienteVIP**, que **tem frete grátis e desconto progressivo**. No fundo, os dois são um tipo de **Cliente** — todos têm nome, documento e endereço e compartilham a mesma base.

Dev sênior: Entendi. Então tem uma parte **igual pra todos** e uma parte **específica** de cada tipo. Guardem isso — cheira a uma base comum com dois tipos que herdam dela.

Cliente: Isso. E o comportamento muda por tipo: olhando o **calcularFrete()** de cada um, pro **ClienteComum** paga o frete normal; já pro **ClienteVIP** tem frete grátis. A **mesma** operação, **resposta diferente conforme o tipo**.

Analista de qualidade: Um ponto sério: o preço do item (registrado na compra) não muda depois; o estoque nunca fica negativo. Ninguém pode mexer nesses dados **por fora** — tem que ser por **operações controladas**. E tem uma regra que **NÃO pode falhar: não pode finalizar um pedido com produto sem estoque**.

Por isso o estado de **Pedido** só anda por operações (novo → pago → enviado → entregue; ou cancelado), nunca "na mão".

Cliente: Agora as ligações. Um cliente faz MUITOS pedidos; um pedido é feito de VÁRIOS itens.

Cliente: Tem uma ligação de **parte-todo que nasce e morre junto: Pedido ♦ ItemPedido**. Cada item pertence a um pedido e some junto se o pedido for cancelado. (pense em **composição** — losango cheio ♦)

Dev sênior: Não confunda com o outro caso: **Categoria ♦ Produto** é só um **agrupamento**. Os produtos existem no catálogo por conta própria; a categoria só os agrupa. Se **Categoria** sumir,

Produto continua existindo — isso é **agregação**
(losango vazio ◇), bem diferente da composição.

Gerente: Fechou. Com isso dá pra desenhar o modelo. Mãos à obra.

Dicas de leitura (pistas escondidas na conversa)

O que apareceu na conversa	Onde isso te leva
"os dois são um tipo de Cliente (mesma base)"	um tipo geral (base) e tipos específicos → herança
"a mesma operação calcularFrete() , resposta muda por tipo"	operação redefinida em cada tipo → polimorfismo
"O preço do item (registrado na compra) não muda depois; o estoque nunca fica negativo"	dado privado + operações → encapsulamento
"não pode finalizar um pedido com produto sem estoque"	invariante: o estado muda só por operação que valida
" Pedido ◆ ItemPedido (nascem e somem juntos)"	composição (losango cheio ◆)
" Categoria ◇ Produto (Produto existe sozinho)"	agregação (losango vazio ◇)
"um cliente faz MUITOS pedidos; um pedido é feito de VÁRIOS itens"	associação com multiplicidade

O que você deve entregar (no draw.io)

Um **diagrama de classes** que contenha, de forma clara:

1. **Abstração** — uma classe **base** que não é criada diretamente (ex.: **Cliente**).
2. **Herança** — **ClienteComum** e **ClienteVIP** ligados à base pelo triângulo vazio →▷.
3. **Encapsulamento** — atributos sensíveis como **privados** (−) e **operações públicas** (+) que os controlam (proteger o valor e o estado da **Pedido**).
4. **Polimorfismo** — a operação **calcularFrete()** declarada na base e **redefinida** em cada tipo.
5. **Os três relacionamentos**, cada um com sua **multiplicidade**:
 - **Associação** (linha/seta): um cliente faz MUITOS pedidos; um pedido é feito de VÁRIOS itens.
 - **Agregação** (losango **vazio** ◇): **Categoria** junta **Produtos** que existem sozinhos.
 - **Composição** (losango **cheio** ◆): **Pedido** é feita de **ItemPedidos** que nascem e morrem com ela.

Lembrete de notação (UML)

Elemento	Como desenhar
Classe abstrata	nome em <i>itálico</i> + « abstract » (não se cria direto)

Elemento	Como desenhar
Operação abstrata	em <i>itálico</i> (cada subtipo redefine)
Visibilidade	– privado · + público · # protegido
Herança	linha com triângulo vazio $\longrightarrow$ apontando para a base
Associação	linha (com seta), com multiplicidade nas pontas
Agregação	losango vazio $\diamond$ no lado do "todo"
Composição	losango cheio $\blacklozenge$ no lado do "todo"

Multiplicidades: **1** (exatamente um), **0..1** (zero ou um), ***** (muitos), **1..*** (um ou mais).

✓ Critério de "pronto"

- ☐ Há uma **classe base abstrata** e os dois tipos herdando dela.
- ☐ Nenhum atributo sensível está público; o valor e o estado da **Pedido** são protegidos.
- ☐ A operação **calcularFrete()** aparece na base e é **redefinida** em cada tipo.
- ☐ Aparecem os **três** relacionamentos: **associação**, **agregação** ($\diamond$) e **composição** ($\blacklozenge$).
- ☐ Você consegue **explicar** por que **Pedido** $\rightarrow$ **ItemPedido** é composição e **Categoria** $\rightarrow$ **Produto** é agregação.
- ☐ Todas as ligações têm **multiplicidade**.

 [Briefing do projeto](#) ·  [Plano de evolução](#)